

Creación de un módulo Niagara N4

Asistente *New Module* de Workbench + configuración del proyecto en IntelliJ IDEA

niagara-research — manual del flujo de trabajo del operador
Ejemplo práctico: ColdRoomPan (módulo de control de cámara frigorífica)

Contents

1 Alcance y requisitos previos	1
2 Parte A — Crear el módulo en Workbench	1
2.1 Abrir el asistente New Module	1
2.2 Paso 1 de 3 — metadatos del módulo y perfiles de runtime	2
2.3 Paso 2 de 3 — dependencias	3
2.4 Paso 3 de 3 — paquetes	4
2.5 Detalle a tener en cuenta: no se puede volver al Paso 1	5
2.6 Finish — el proyecto generado	6
3 Parte B — Abrir el proyecto en IntelliJ IDEA	7
3.1 Iniciar IntelliJ IDEA	7
3.2 Abrir la carpeta del módulo	7
3.3 Confiar en el proyecto	9
4 Parte C — Configurar el proyecto	10
4.1 Establecer el SDK y el nivel de lenguaje	10
4.2 Editar <code>gradle.properties</code>	11
4.3 Las dos ediciones dependientes de la versión	12
4.4 Resultado — entorno listo	12
5 Parte D — Programación: tareas de Gradle y archivos clave	13
5.1 Abrir la ventana de herramientas de Gradle (ícono del elefante)	13
5.2 Las tareas que importan: build, clean, slotomatic	14
5.3 Archivos clave del módulo a tener presentes	15
5.4 Material de referencia	16
6 Resumen	16

1 Alcance y requisitos previos

Este manual documenta, paso a paso, cómo un operador crea el **esqueleto** vacío de un módulo Niagara N4 con el asistente *New Module* de Workbench, y luego abre y configura ese proyecto en **IntelliJ IDEA** para dejarlo listo para programar. Es la vía “manual” — el asistente genera un proyecto `gradle-niagara` estándar que después se edita en el IDE.

El ejemplo práctico que se emplea a lo largo del documento es el módulo usado para la lógica de la cámara frigorífica: ColdRoomPan (símbolo CRP, proveedor Angeles).

Requisitos previos.

- Niagara Workbench (aquí: *Optimizer Supervisor* N4.14.0.162).
- IntelliJ IDEA (aquí: 2026.1.1).
- Un **JDK 8** completo (aquí: Azul Zulu 8) — las compilaciones de Niagara requieren un JDK real, no solo un JRE.
- Opcionalmente Node.js, únicamente si el módulo va a compilar partes de JavaScript (UX).

Nota. Esto produce exactamente el tipo de proyecto `gradle-niagara` que describe el runbook `docs/module-dev-workflow.md` del repositorio; una vez que está abierto y configurado, se continúa con el ciclo *editar* → *build* → *firmar* → *desplegar* → *probar*.

2 Parte A — Crear el módulo en Workbench

2.1 Abrir el asistente New Module

En Workbench, abra el menú **Tools** y elija **New Module** (Figura 1).

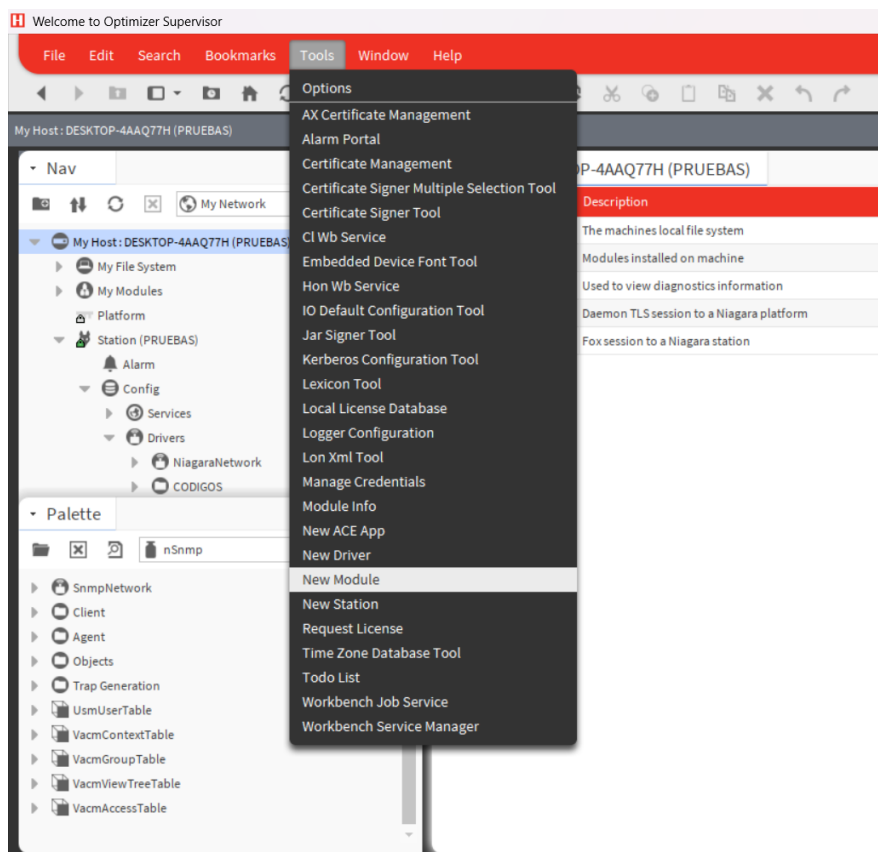


Figure 1: Workbench Tools → New Module.

2.2 Paso 1 de 3 — metadatos del módulo y perfiles de runtime

El asistente se abre en el *Step 1 of 3*. El primer campo es la ruta **Create Module under Directory** — la carpeta donde se generará el proyecto (Figura 2). En este ejemplo: `C:/modulos_niagara_n4/Cliente/Leon-Guanajuato/Paccadia`.

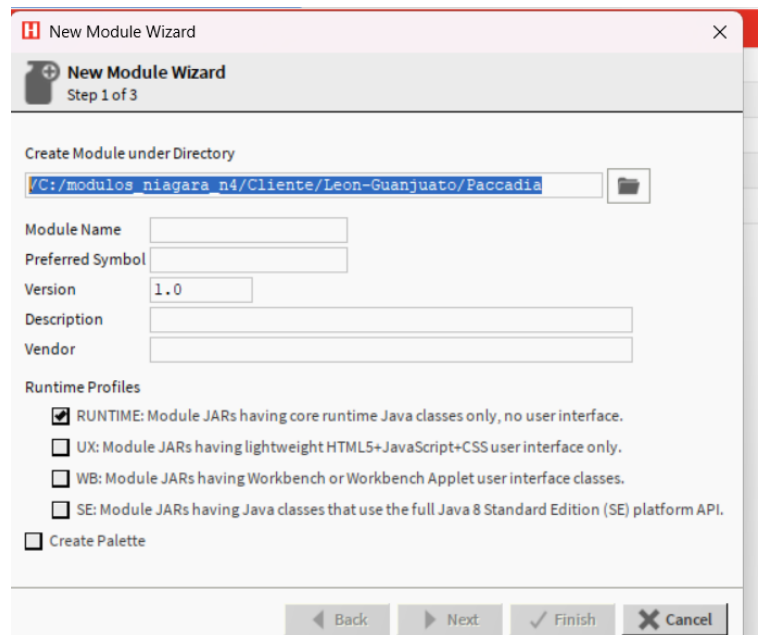


Figure 2: Step 1 of 3, vacío. La ruta del directorio se selecciona primero.

Complete los campos restantes (Figura 3):

- **Module Name** — ColdRoomPan. El nombre técnico del módulo.
- **Preferred Symbol** — CRP. Prefijo/símbolo corto del módulo.
- **Version** — 1.0.
- **Description** — Logic Cold Room Paccadia.
- **Vendor** — Angeles.
- **Runtime Profiles** — qué perfiles JAR incluye el módulo (véase más abajo).
- **Create Palette** — cuando está marcado, genera un archivo de paleta para que los componentes del módulo puedan arrastrarse desde la paleta de Workbench.

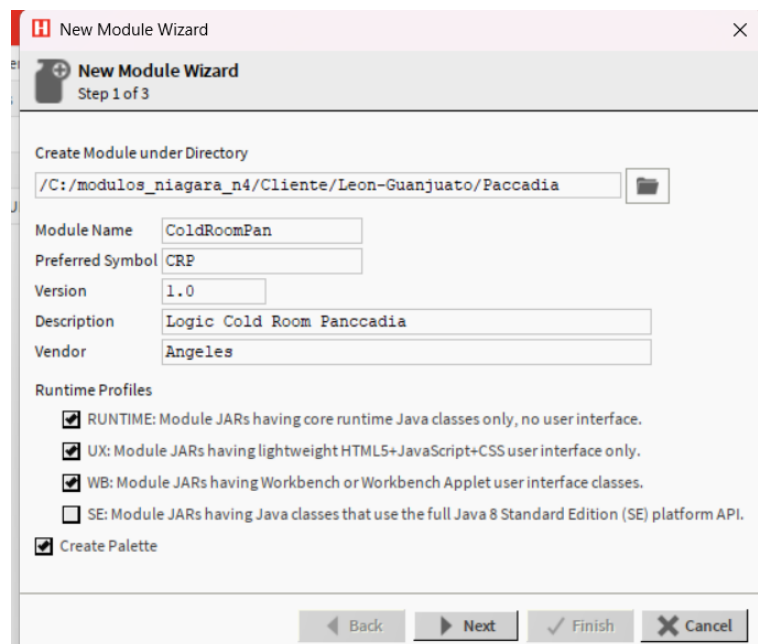


Figure 3: Step 1 completado, con los perfiles RUNTIME + UX + WB y *Create Palette* marcado (primer intento).

Runtime Profiles — qué significa cada uno.

RUNTIME (rt)

Solo clases Java del runtime central, sin interfaz de usuario.

UX Interfaz de usuario ligera en HTML5 + JavaScript + CSS.

WB Clases de interfaz de usuario de Workbench (o del applet de Workbench).

SE Clases Java que usan la API completa de la plataforma Java 8 Standard Edition.

Nota. La opción recomendada para este proyecto es **RUNTIME (rt) puro**. Los perfiles *UX* y *WB* solo son necesarios si desea una interfaz de navegador o una vista de Workbench dentro del módulo — no se requieren para la lógica de control. Empezar solo con RT mantiene el módulo pequeño y simple; UX/WB se pueden añadir después si realmente se quiere una interfaz de Workbench.

2.3 Paso 2 de 3 — dependencias

Al hacer clic en **Next** se avanza al *Step 2 of 3, Select Dependencies*. De forma predeterminada, el módulo depende de **nre** y **baja** (ambos Tridium 4.14), mostrados en la Figura 4. Use **Add** para incorporar más módulos (p. ej. **control**, **kitControl**, **alarm**, **history**, **schedule**) desde el selector de la Figura 5.

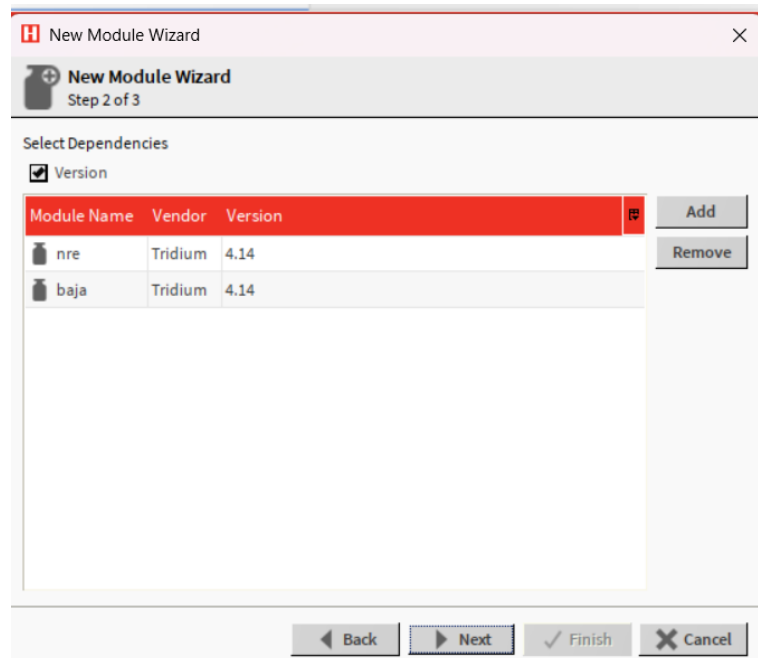


Figure 4: Step 2 of 3 — dependencias predeterminadas **nre** y **baja**.

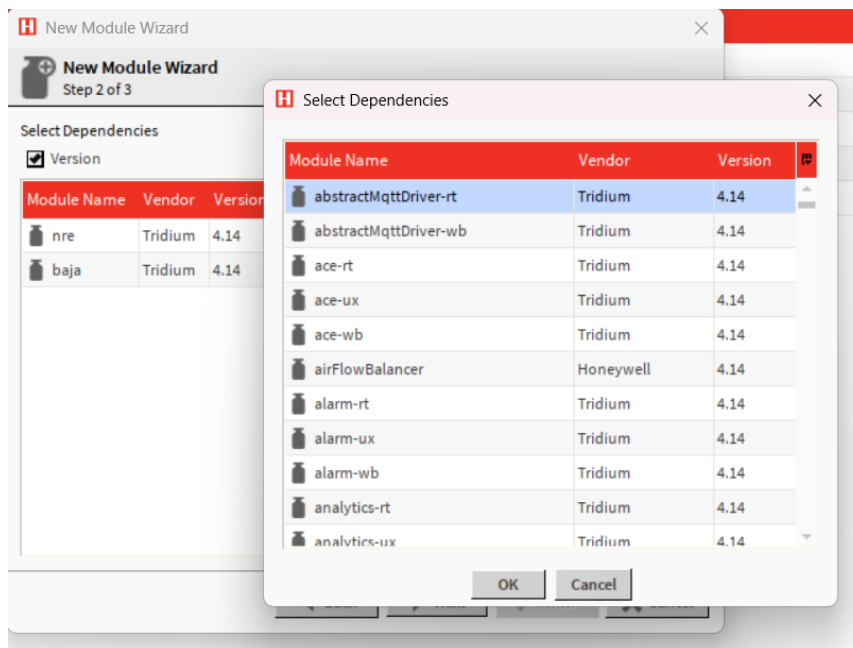


Figure 5: El selector *Select Dependencies* (Add).

Nota. Las dependencias también se pueden añadir *más adelante*, directamente en la configuración de compilación del módulo, así que no hay problema en dejar los valores predeterminados aquí y añadir lo que la lógica necesite durante el desarrollo.

2.4 Paso 3 de 3 — paquetes

Next avanza al *Step 3 of 3, Add Packages*. El asistente propone un paquete Java por cada perfil de runtime seleccionado (Figura 6): `com.angeles.ColdRoomPan` (rt), `com.angeles.ColdRoomPan.ui` (wb) y `com.angeles.ColdRoomPan.ux` (ux). El prefijo del paquete sigue al proveedor (`com.angeles`).

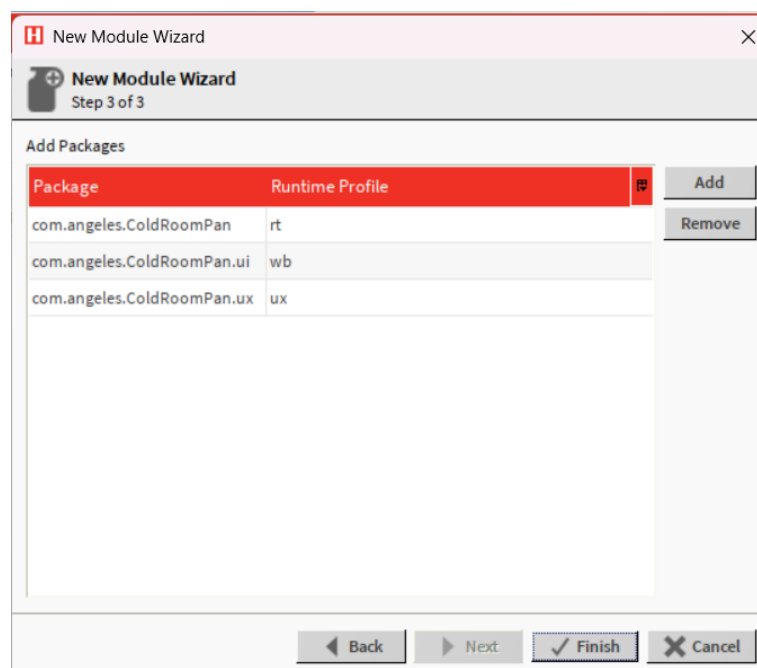


Figure 6: Step 3 of 3 — un paquete por perfil (rt/wb/ux) en el primer intento.

2.5 Detalle a tener en cuenta: no se puede volver al Paso 1

Una vez que pulsa **Next** pasando el Paso 1, el botón **Back** ya no regresa al Paso 1. Para cambiar los perfiles de runtime (aquí: quitar UX/WB y conservar solo RT) debe **cancelar y volver a iniciar el asistente**.

La segunda ejecución conserva únicamente **RUNTIME** más *Create Palette* (Figura 7), lo que produce un solo paquete `com.angeles.ColdRoomPan` con el perfil `rt` (Figura 8).

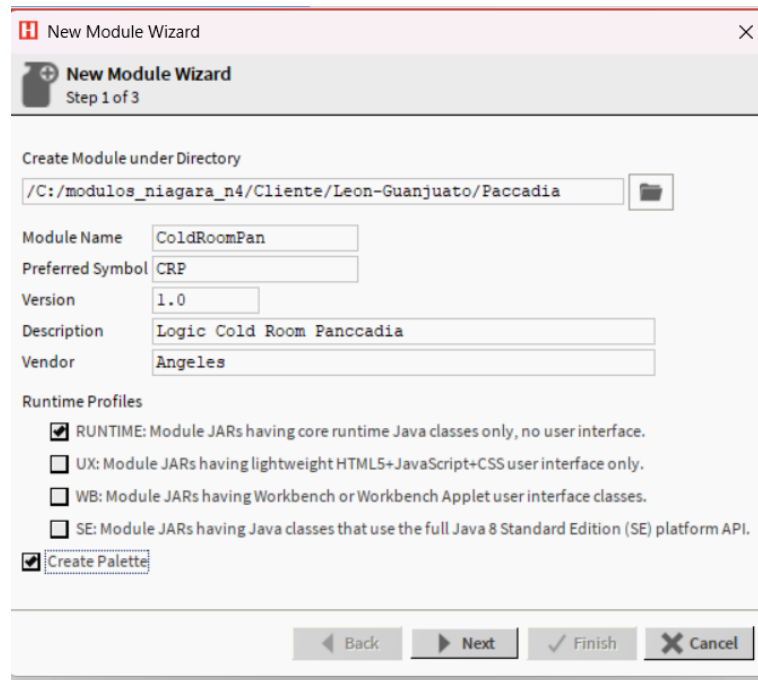


Figure 7: Segunda ejecución — Step 1 solo con RUNTIME + Create Palette.

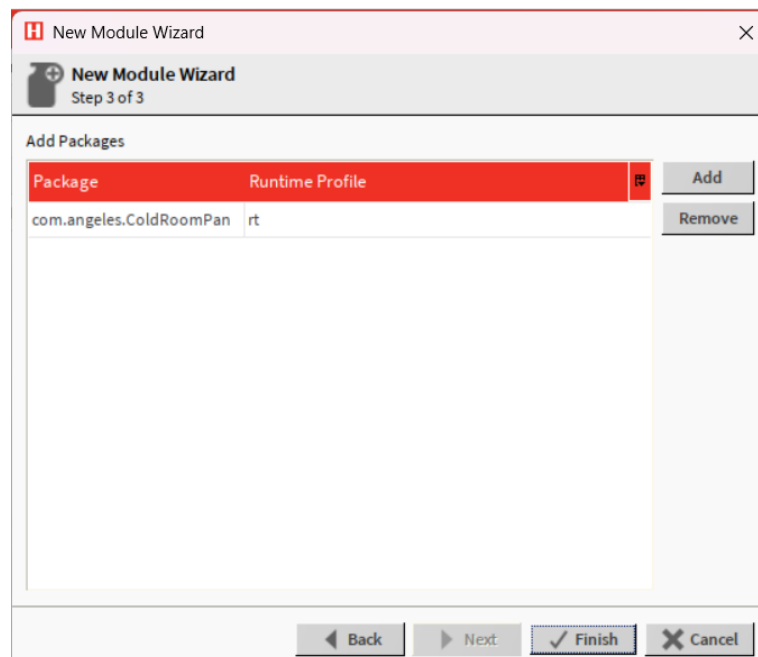


Figure 8: Segunda ejecución — Step 3 con un único paquete `rt`.

2.6 Finish — el proyecto generado

Al pulsar **Finish** se crea el proyecto. Al explorar la carpeta de destino en Workbench se muestra el esqueleto generado (Figura 9):

- ColdRoomPan/ — el directorio de código fuente del módulo.
- gradle/ — los archivos de soporte del wrapper de Gradle.
- build.gradle.kts, settings.gradle.kts — scripts de compilación de Gradle.
- gradle.properties — configuración de compilación (rutas, JDK) — se edita más adelante.
- gradlew, gradlew.bat — los lanzadores del wrapper de Gradle.

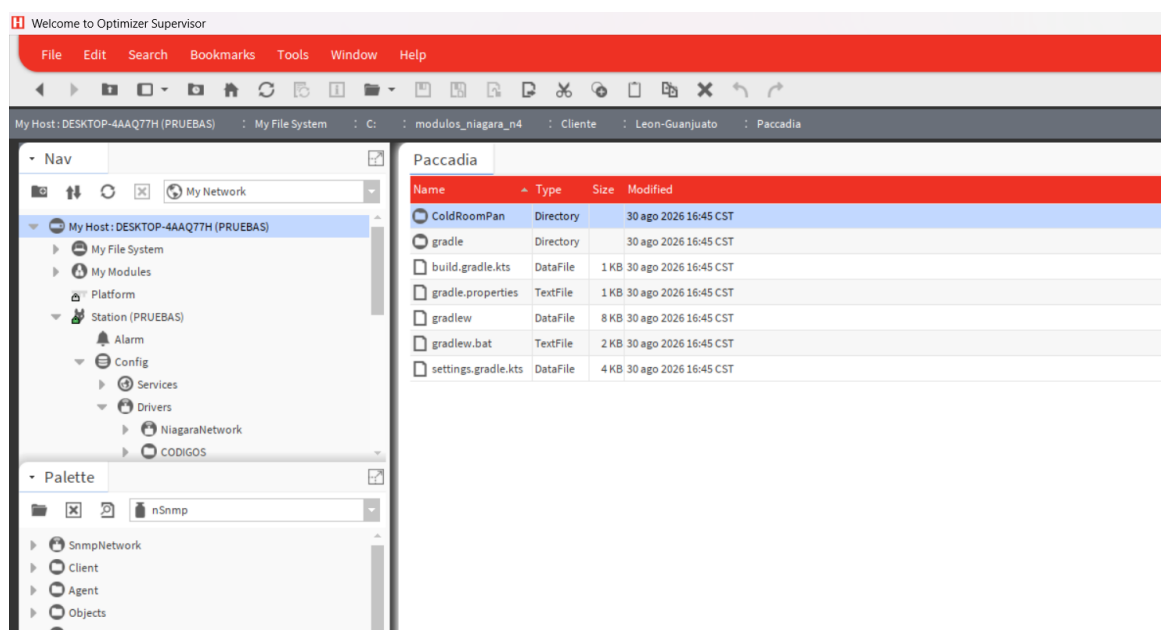


Figure 9: Los archivos del proyecto generado en el directorio de destino (Paccadia).

En este punto el esqueleto existe; lo que queda es abrirlo en el IDE y apuntar la compilación a la instalación de Niagara y al JDK correctos.

3 Parte B — Abrir el proyecto en IntelliJ IDEA

3.1 Iniciar IntelliJ IDEA

Abra IntelliJ IDEA (Figura 10).

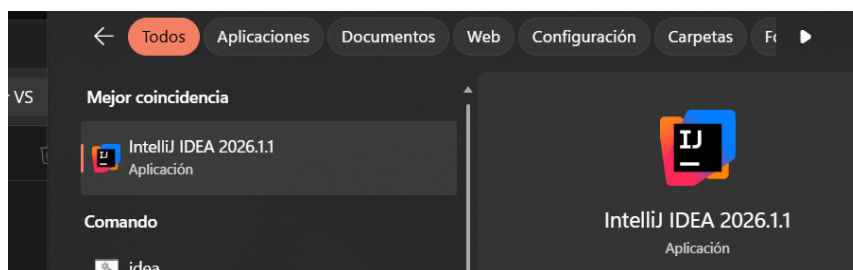


Figure 10: Inicio de IntelliJ IDEA 2026.1.1.

3.2 Abrir la carpeta del módulo

En la pantalla de bienvenida (Figuras 11 y 12) haga clic en **Open**, luego navegue hasta la carpeta del módulo Paccadia y selecciónela (Figura 13) y pulse **Select Folder**.

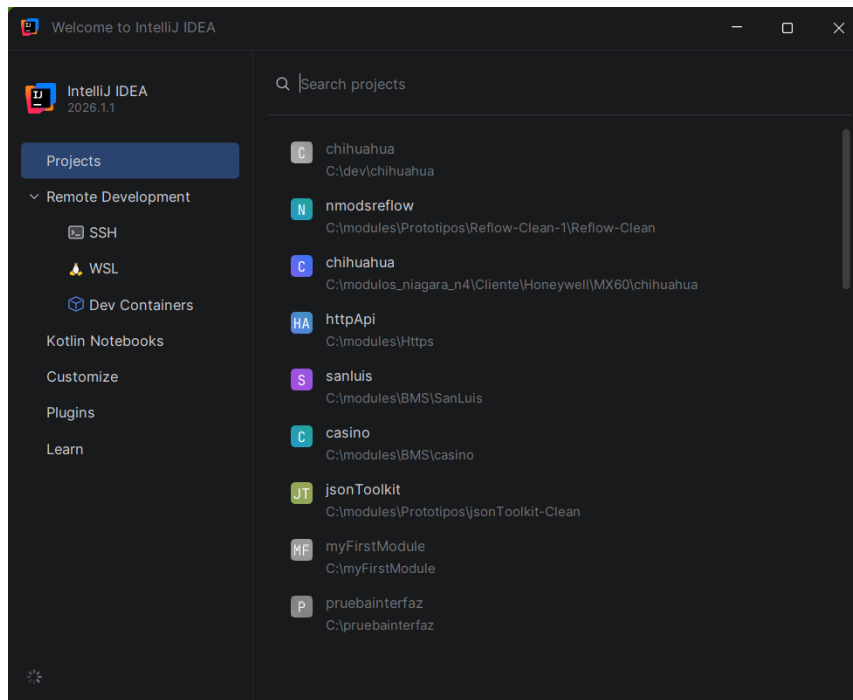


Figure 11: Pantalla de bienvenida de IntelliJ — proyectos recientes.

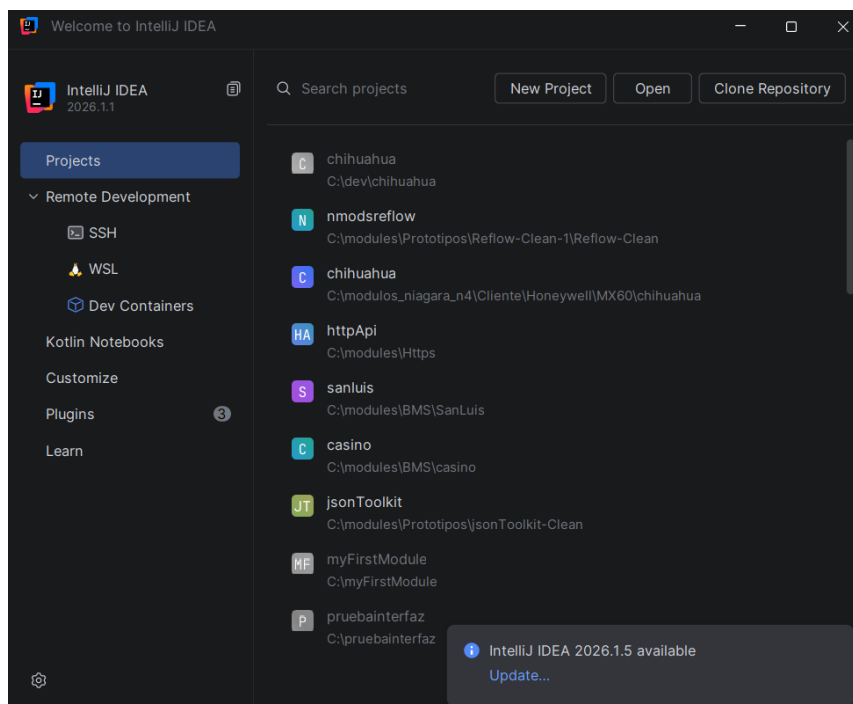


Figure 12: Pantalla de bienvenida — *New Project / Open / Clone Repository*.

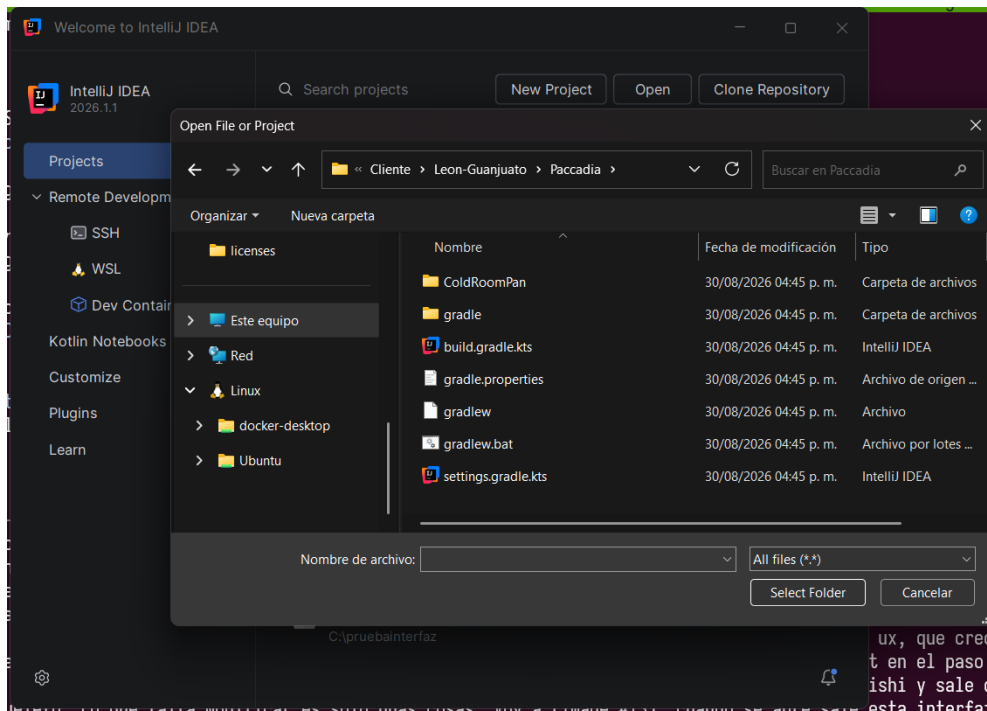


Figure 13: *Open File or Project* — seleccione la carpeta Paccadia.

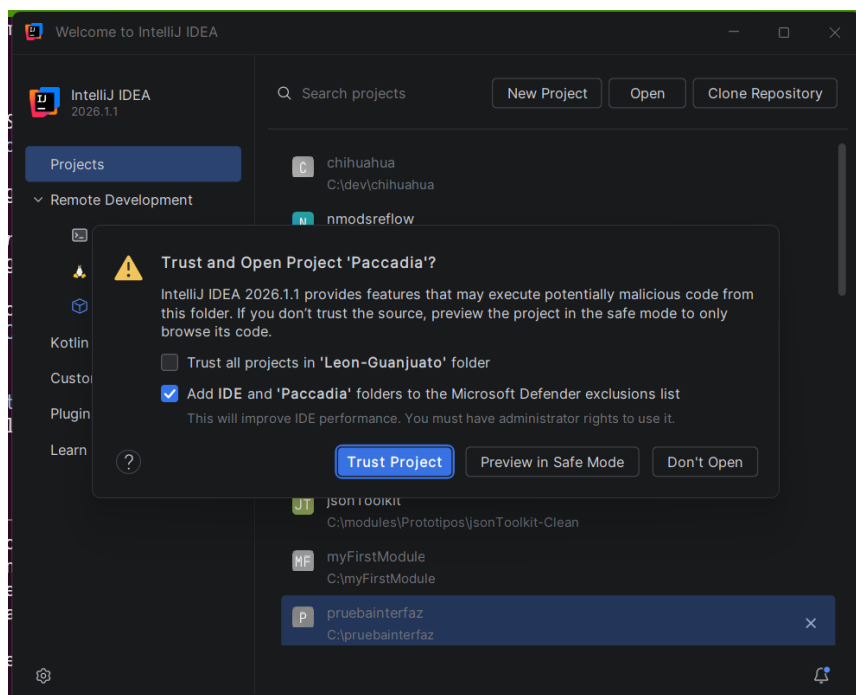


Figure 14: Al hacer clic en **Open** para buscar el proyecto.

3.3 Confiar en el proyecto

IntelliJ pregunta si desea confiar en la carpeta (Figura 15). Elija **Trust Project**. (El cuadro de diálogo opcionalmente añade las carpetas del IDE y del proyecto a la lista de exclusiones de Microsoft Defender para mejorar el rendimiento.)

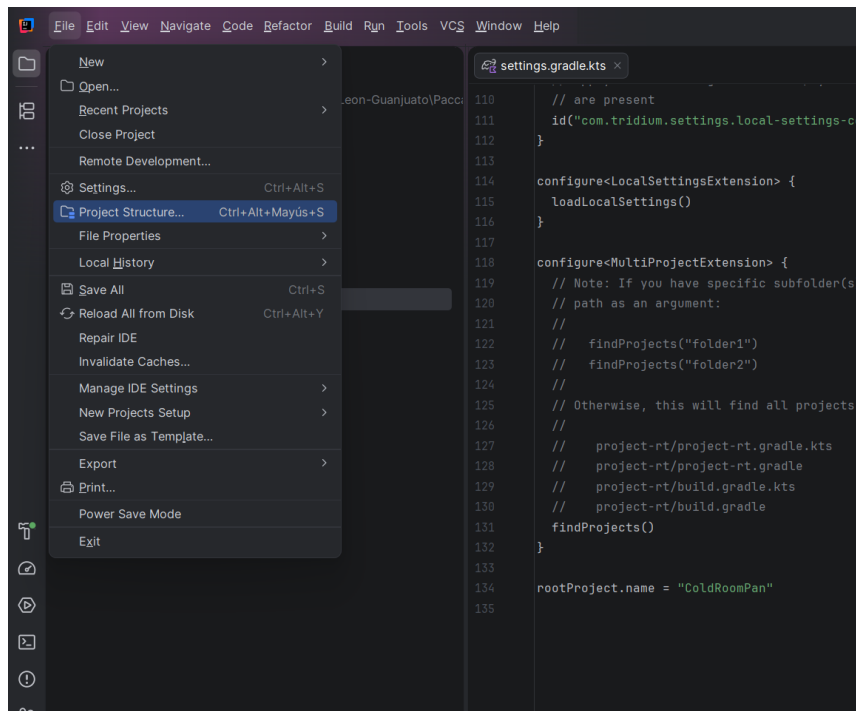


Figure 15: Trust and Open Project 'Paccadia'? → Trust Project.

Tras confiar, IntelliJ comienza a importar el proyecto de Gradle (Figura 16); espere a que termine la importación.

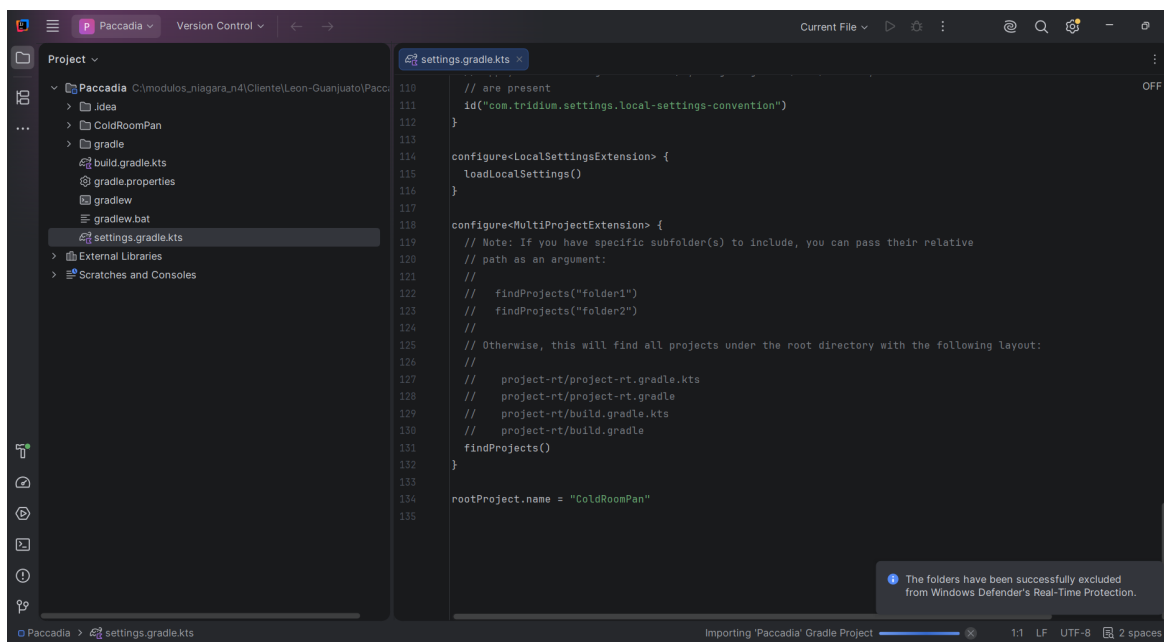


Figure 16: IntelliJ importando el proyecto de Gradle Paccadia.

4 Parte C — Configurar el proyecto

4.1 Establecer el SDK y el nivel de lenguaje

Abra **File** → **Project Structure** (Figura 17).

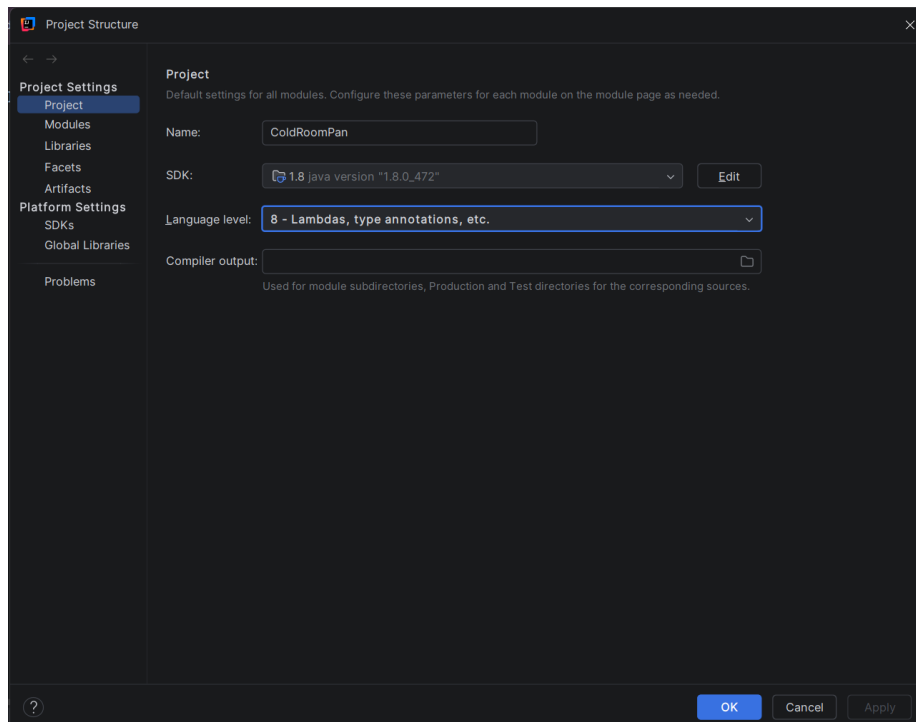


Figure 17: File → Project Structure.

En **Project Settings** → **Project** (Figura 18) establezca los dos valores que requiere Niagara N4:

- **SDK:** 1.8 (aquí 1.8.0_472).
- **Language level:** 8 - Lambdas, type annotations, etc.

Aplique y espere un momento a que los cambios surtan efecto.

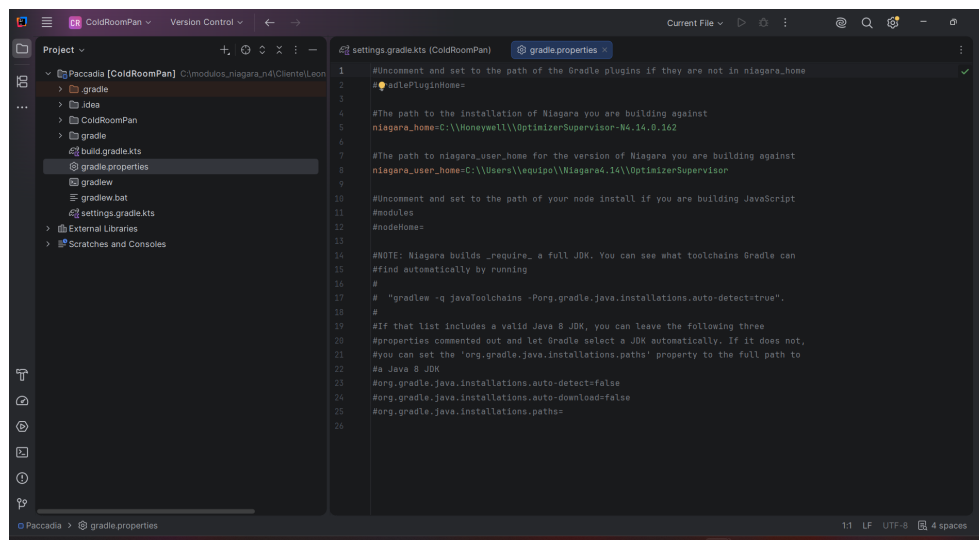


Figure 18: Project Structure — SDK 1.8 y nivel de lenguaje 8.

4.2 Editar gradle.properties

Abra `gradle.properties` (Figura 19). Este archivo indica a la compilación qué instalación de Niagara y qué JDK usar.

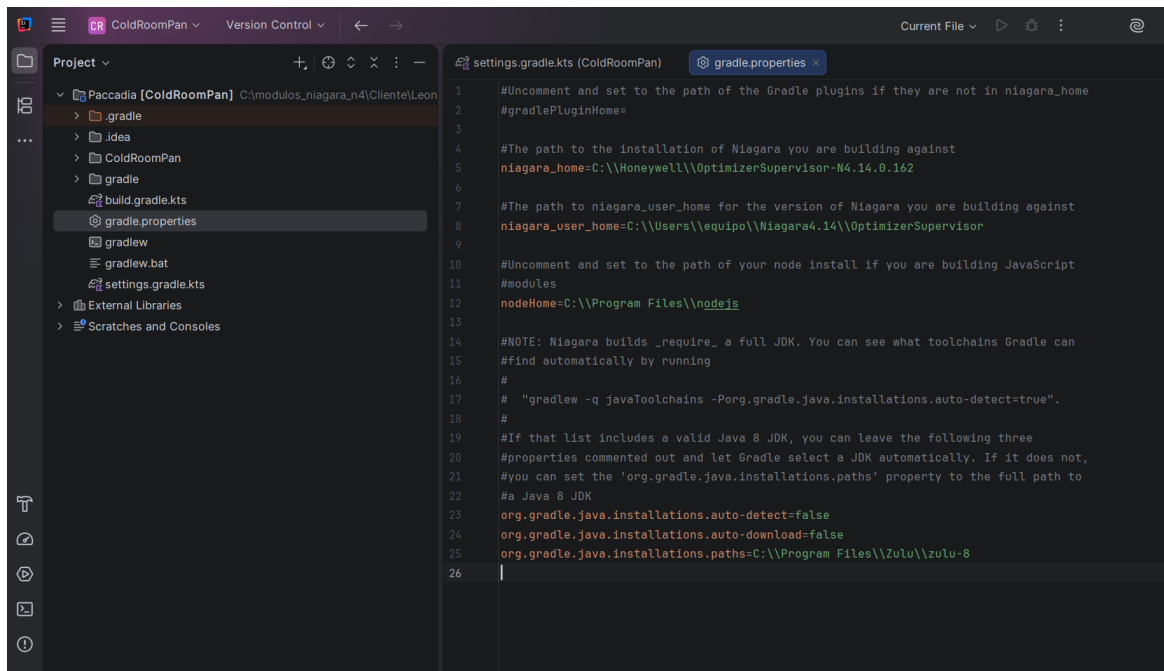


Figure 19: gradle.properties antes de editar (líneas del JDK comentadas).

Las claves relevantes, tal como se generan:

```
# The path to the installation of Niagara you are building against
niagara_home=C:\\Honeywell\\OptimizerSupervisor-N4.14.0.162

# The path to niagara_user_home for the version you are building against
niagara_user_home=C:\\Users\\equipo\\Niagara4.14\\OptimizerSupervisor

# Uncomment and set to the path of your node install if building JavaScript
#nodeHome=

#org.gradle.java.installations.auto-detect=false
#org.gradle.java.installations.auto-download=false
#org.gradle.java.installations.paths=
```

Aplice las ediciones (descomente y establezca la cadena de herramientas del JDK, y Node si se usa):

```
niagara_home=C:\\Honeywell\\OptimizerSupervisor-N4.14.0.162
niagara_user_home=C:\\Users\\equipo\\Niagara4.14\\OptimizerSupervisor

nodeHome=C:\\Program Files\\nodejs

org.gradle.java.installations.auto-detect=false
org.gradle.java.installations.auto-download=false
org.gradle.java.installations.paths=C:\\Program Files\\Zulu\\zulu-8
```

Nota. Fijar `org.gradle.java.installations.paths` al JDK Zulu 8 y desactivar *auto-detect* / *auto-download* garantiza que la compilación use la cadena de herramientas Java 8 correcta en lugar de la que Gradle pudiera elegir.

4.3 Las dos ediciones dependientes de la versión

Cuando apunte a una versión *diferente* de Niagara, cambian dos cosas:

1. `gradle.properties` — actualice `niagara_home` (y `niagara_user_home`) a la ruta de esa instalación de Niagara.
2. `settings.gradle.kts` — actualice la versión de Gradle a la que compila esa versión de Niagara.

4.4 Resultado — entorno listo

Una vez que el proyecto de Gradle termina de importarse con el SDK y las rutas correctos, la ventana de herramientas de Gradle muestra el proyecto y su subproyecto `ColdRoomPan-rt` con sus tareas (Figura 20). El entorno ya está listo para empezar a programar el módulo.

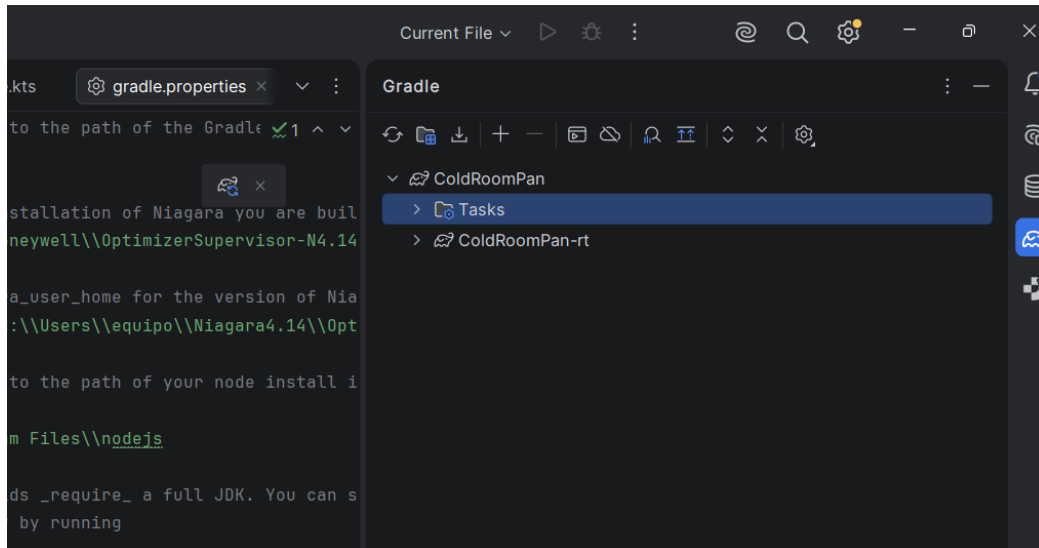


Figure 20: Ventana de herramientas de Gradle — `ColdRoomPan` y `ColdRoomPan-rt` cargados.

5 Parte D — Programación: tareas de Gradle y archivos clave

Una vez que el proyecto está abierto y configurado, se programa el módulo y se dirige la compilación desde la ventana de herramientas de **Gradle**. Recuerde que la **versión de Gradle depende de la versión de Niagara** (se establece en `settings.gradle.kts`).

5.1 Abrir la ventana de herramientas de Gradle (icono del elefante)

Abra el panel de Gradle desde el icono del elefante en el borde derecho (Figuras 21 y 22). Lista el proyecto raíz `ColdRoomPan`, sus **Tasks** y el subproyecto `ColdRoomPan-rt`.

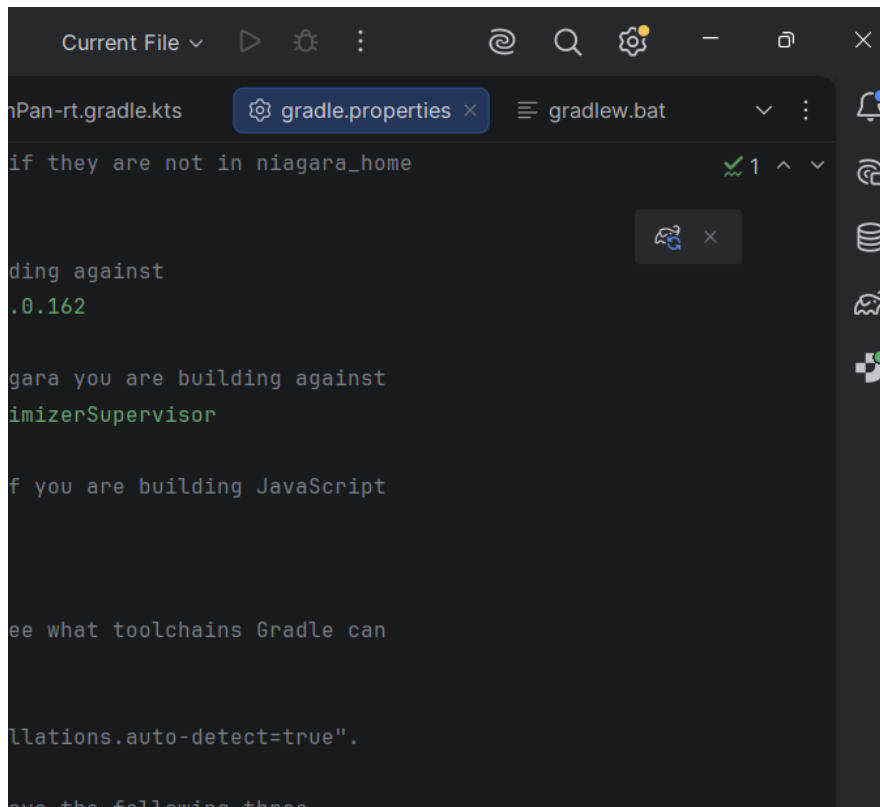


Figure 21: Apertura de la ventana de herramientas de Gradle.

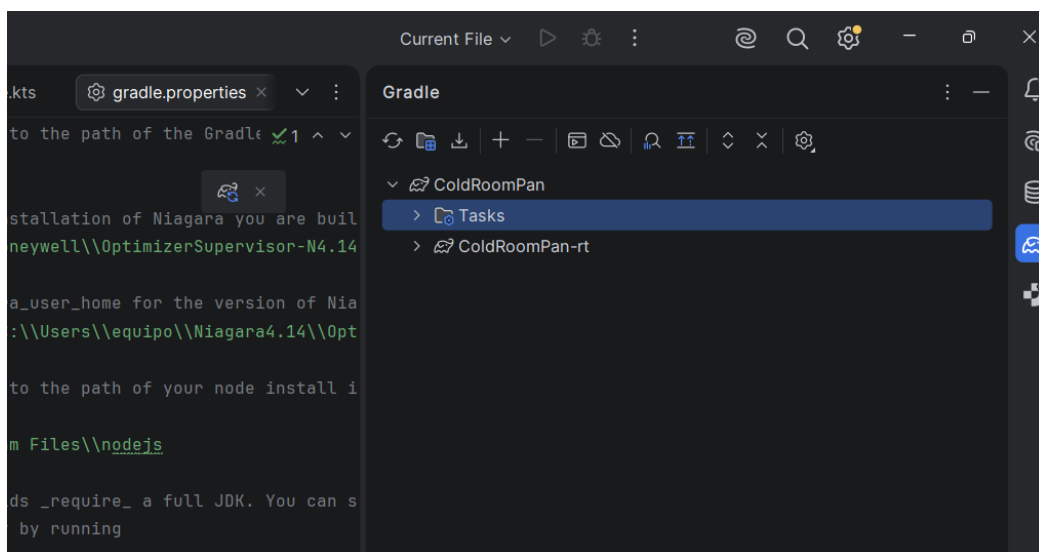


Figure 22: Proyecto raíz de Gradle ColdRoomPan con sus *Tasks* y la parte ColdRoomPan-rt.

5.2 Las tareas que importan: build, clean, slotomatic

Expanda **Tasks** (Figura 23). Importan dos grupos:

- **build** — assemble, build, buildDependents, buildNeeded, classes, clean, jar, moduleTestClasses, moduleTestJar, testClasses. Las dos que más se usan son build y clean.
- **niagara** — migrateSlotomatic y slotomatic. slotomatic es la tarea de generación de código.

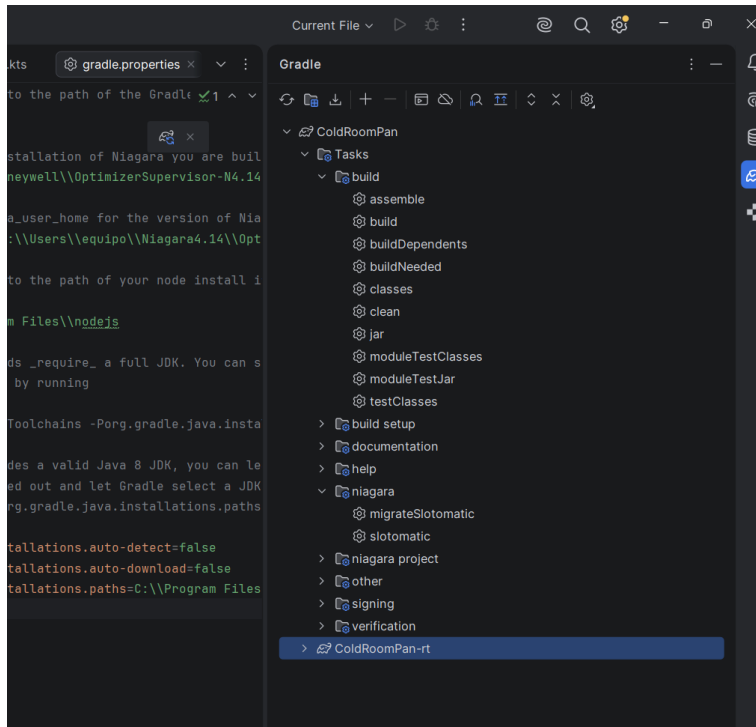


Figure 23: Tasks expandido: las tareas del engranaje *build* y el grupo *niagara* con *slotomatic*.

Nota. Regla de Slotomatic (documentada en docs/module-dev-workflow.md §1.3/§3 y docs/module-best-practices.md §5.2): ejecute `:slotomatic` solo cuando cambie una anotación `@Niagara*`. Escribe la región de slots AUTO de la clase; **nunca** edite a mano dentro de los marcadores AUTO. Una región AUTO obsoleta provoca un fallo de compilación/ejecución. Ciclo típico: `clean` → `slotomatic` (solo al cambiar una anotación) → `build`.

5.3 Archivos clave del módulo a tener presentes

La parte `ColdRoomPan-rt` contiene los archivos descriptores de Niagara (Figura 24):

`module.palette`

BOG de componentes para arrastrar y soltar desde la paleta de Workbench. Inclúyalo en cualquier módulo que exporte componentes reutilizables (sin gate, lazy, tolerante a fallos). Documentado: `module-best-practices.md` [B634].

`module-permissions.xml`

Un *manifiesto de solicitud de permisos del Java Security Manager*. Declara qué capacidades restringidas del host necesita el *código* del propio módulo — sockets de red, acceso a archivos, bibliotecas nativas, autenticación, hora del sistema, backups, keystore — para que el módulo pueda ejecutar operaciones protegidas sin un `AccessControlException`. Raíz `<permissions>`, con bloques `<niagara-permission-groups type="station|workbench|all">` que contienen entradas `<req-permission>` / `<opt-permission>` (`<name>`, `<purposeKey>`, `<parameters>` opcionales como hosts/puertos). **No** controla cómo se ve el módulo y **no** es RBAC por usuario. El devkit lo genera como una plantilla totalmente comentada (sin permisos adicionales solicitados); el módulo `chihuahua` incluye exactamente esa plantilla. Doc: `devguide security/requestingPermissions.t`. §1, §2.1.1, §2.2; referencia de build `build.txt:160`.

`module.lexicon`

Cadenas de localización del módulo.

`niagara-module.xml` / `module-include.xml`

La lista exportada de `<type>` que lee Slotomatic.

`ColdRoomPan-rt.gradle.kts`

El script de compilación de la parte: `moduleManifest (moduleName, runtimeProfile=rt)`,

dependencies (nre(":nre"), api(":baja"), moduleTestImplementation(":test-wb")), y la tarea bajadoc (includePackage("com.angeles.ColdRoomPan")).

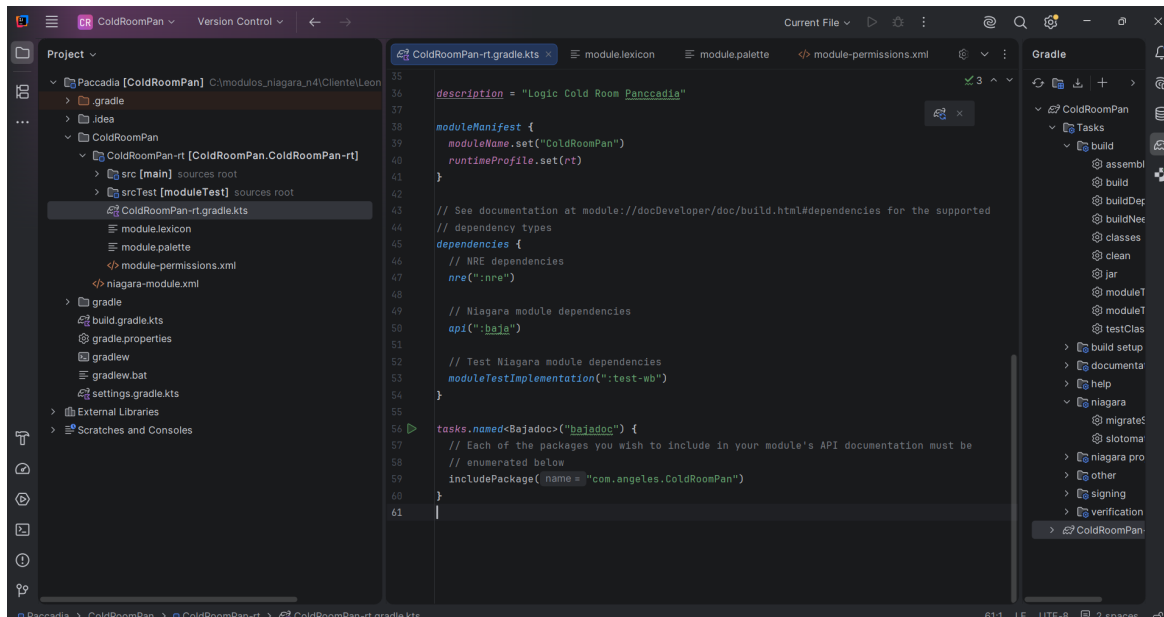


Figure 24: La parte ColdRoomPan-rt: module.palette, module-permissions.xml, module.lexicon, niagara-module.xml, y ColdRoomPan-rt.gradle.kts.

5.4 Material de referencia

- Herramientas locales: \\wsl.localhost\Ubuntu\home\cristian\modulos_niagara_n4\niagara-tools
- Ejemplo práctico real para estudiar la estructura completa de un módulo: \\wsl.localhost\Ubuntu\home\cristian\modulos_niagara_n4\Cliente\Honeywell\MX60\chihuahua
- Runbooks del repositorio que documentan esta parte: docs/module-dev-workflow.md y docs/module-best-practices.md.

6 Resumen

1. Workbench: Tools → New Module.
2. Step 1: ruta + Module Name, Preferred Symbol, Version, Description, Vendor; elija solo **RUNTIME**; marque **Create Palette**. (Recuerde: no se puede volver al Step 1.)
3. Step 2: dependencias (predeterminadas **nre**, **baja**; añada más aquí o más adelante).
4. Step 3: un paquete **rt com.<vendor>.<Module>**; Finish.
5. IntelliJ: **Open** la carpeta generada → **Trust Project** → espere la importación de Gradle.
6. **Project Structure**: SDK 1.8, nivel de lenguaje 8.
7. **gradle.properties**: establezca **niagara_home**, **niagara_user_home**, **nodeHome** (si hace falta), y fije la cadena de herramientas del JDK Zulu 8.
8. Por versión de Niagara: ajuste **niagara_home** en **gradle.properties** y la versión de Gradle en **settings.gradle.kts**.

Valores de ejemplo usados en este manual.

Campo	Valor
Module Name	ColdRoomPan
Preferred Symbol	CRP
Version	1.0
Description	Logic Cold Room Panccadia
Vendor	Angeles
Package (rt)	com.angeles.ColdRoomPan
Directory	C:/modulos_niagara_n4/Cliente/Leon-Guanajuato/Paccadia
Niagara home	C:/Honeywell/OptimizerSupervisor-N4.14.0.162
JDK	Zulu 8 (C:/ProgramFiles/Zulu/zulu-8)